

High Performance and Distributed Computing for Big Data

Unit 3: Cloud Systems and Big Data Management

Session 5 - AWS Lambda

Francesc Solsona Tehas francesc.solsona@udl.cat

Universitat Rovira i Virgili and Universitat de Lleida

Cloud functions

Cloud functions enable **serverless** computing, allowing developers to run code without provisioning or managing servers.

Example

Automatically processing patient data uploads, triggering real-time alerts, and updating medical dashboards without infrastructure management.

How Serverless Functions Work



Figure 1: Serverless model



AWS Lambda



CLOUD FUNCTIONS



Azure
Functions

Figure 2: Serverless providers

AWS Lambda

Introduction to AWS Lambda

What is AWS Lambda?

- Event-driven, serverless computing service.
- Runs code in response to triggers.

Key benefits

- Automatic scaling and high availability.
- Pay-per-use billing model.

Common use cases

- Real-time file processing (e.g., medical imaging).
- Backend for web and mobile health applications.

Supported languages

- Python, Node.js, Java, Go, Ruby, .NET, and more.

AWS Lambda architecture overview



Figure 3: Lambda Architecture

AWS Lambda architecture overview

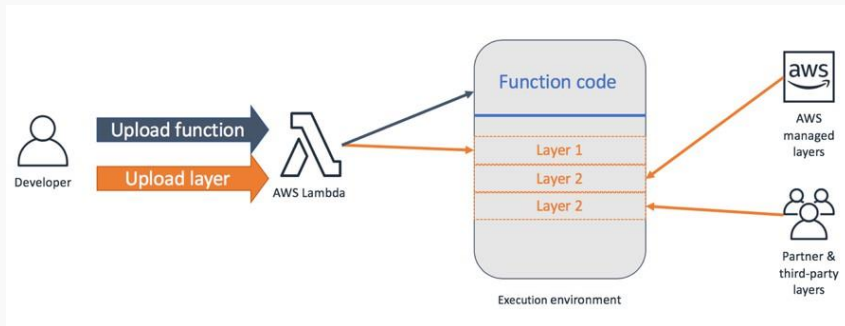


Figure 4: Lambda Architecture

How AWS Lambda compares to EC2

AWS EC2

- Full control over infrastructure (turn it on/off, upgrading).
- Manual scalability management (want more? You have to add more).
- Fixed cost for uptime.

AWS Lambda

- No infrastructure management (serverless).
- Automatic scalability (from zero to thousands).
- Pay for actual execution time.

Use cases for AWS Lambda

- **Real-time data processing:** Lambda can analyze sensor data from wearable devices to identify irregular patterns.
- **Data aggregation:** Lambda can collect data from multiple clinical trial sites, aggregating information on patient outcomes, adverse events, and treatment efficacy preparing a dashboard for the researchers.
- **Data validation:** Lambda can validate lab results (such as blood tests) by checking for outliers or inconsistencies. For example, abnormal values outside the expected range may require further investigation. Alerts can be sent to the lab technician or the patient's healthcare provider.
- **Image processing:** Lambda can process images to identify patterns or anomalies.

Example: Counting cells at scale

Scenario

Imagine a lab that generates a large collection of cell images every time they run an experiment. Once the experiment is done, they want to know the number of cells in each image to analyze the results but they want this process to be automated and immediate.

Workflow

1. A lab uploads a collection of cell images to S3.
2. S3 events trigger Lambda functions (one event per image, one function per event).
3. Lambda functions process the images and count the cells.
4. Results are stored in S3.

Example: Counting cells at scale

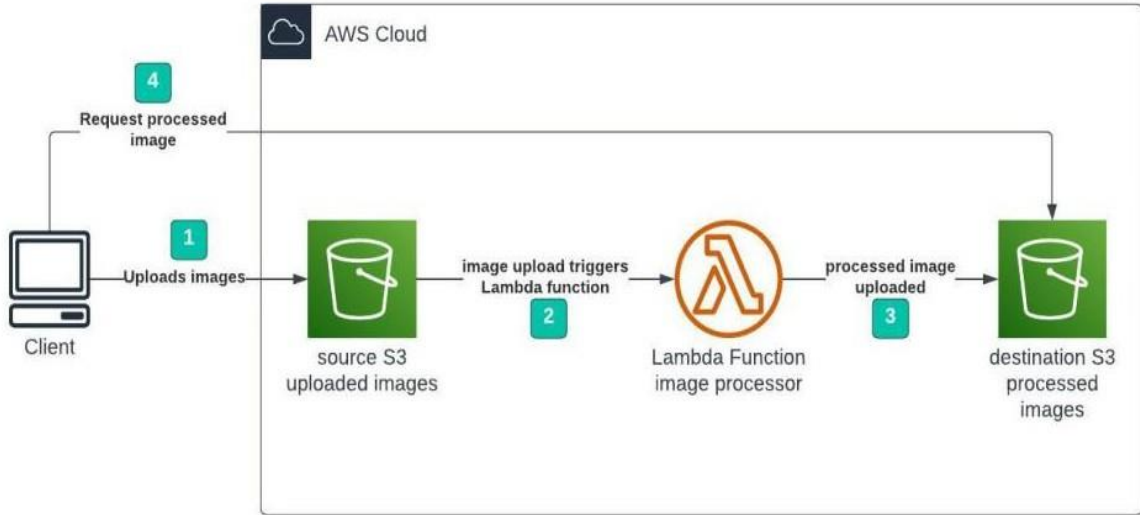


Figure 5: Health Data Flow

Lab: Counting cells at scale

Pre-requisites

- A machine with AWS Credentials configured and Python 3.13 installed. (I am going to use the EC2 instance we created in the previous unit together with `uv` for managing python versions).
- The cell images downloaded and extracted. The image dataset for cell counting can be downloaded by clicking [here](#).

Goal

- Upload a collection of cell images to an S3 bucket and trigger a Lambda function to count the cells in each image. The lambda will store the results in another S3 bucket.

Steps

1. Create the buckets.
2. Create the Lambda function.
3. Add a trigger to the Lambda function.
4. Write the Lambda function code.
5. Create and publish a Lambda layer with the dependencies.
6. Upload the images to the input bucket.
7. Check the results in the output bucket and verify the Lambda logs.

Step 1: Create the buckets

As we did in the previous session, we are going to visit the S3 service in the AWS console and create two buckets, one for the input images and another for the output results:

- Input bucket: `medical-images-raw-[YOUR-NAME]`
- Output bucket: `medical-images-processed-[YOUR-NAME]`

In my case that will be `medical-images-raw-ferran-aran` and `medical-images-processed-ferran-aran`.

S3 Bucket names

Remember S3 bucket names must be unique across all AWS accounts. If you get an error when creating the bucket, try a different name (e.g., add a random number at the end).

Step 2: Create the Lambda function

We are going to search for `lambda` in the AWS Console as usual and click on the first result.

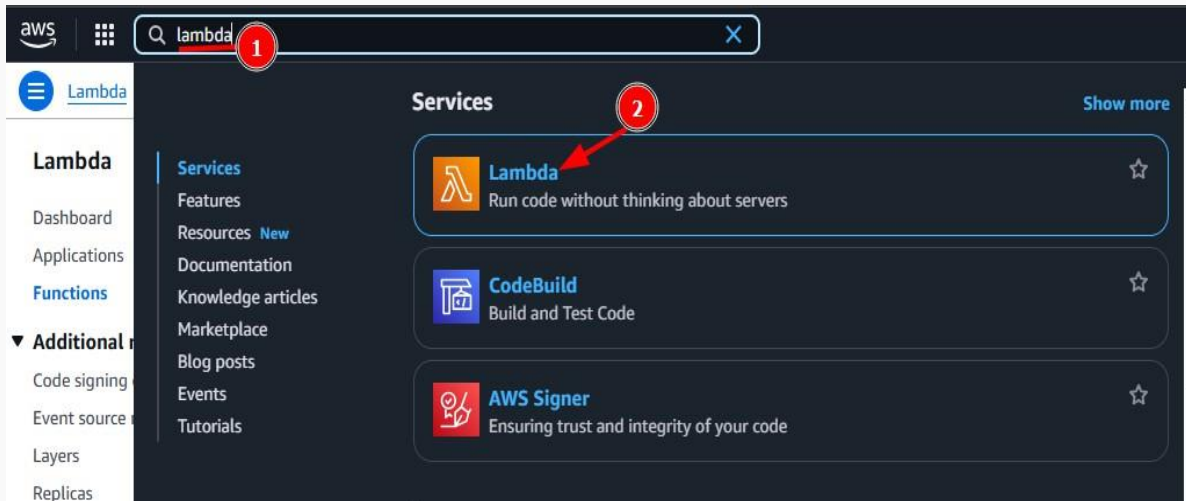
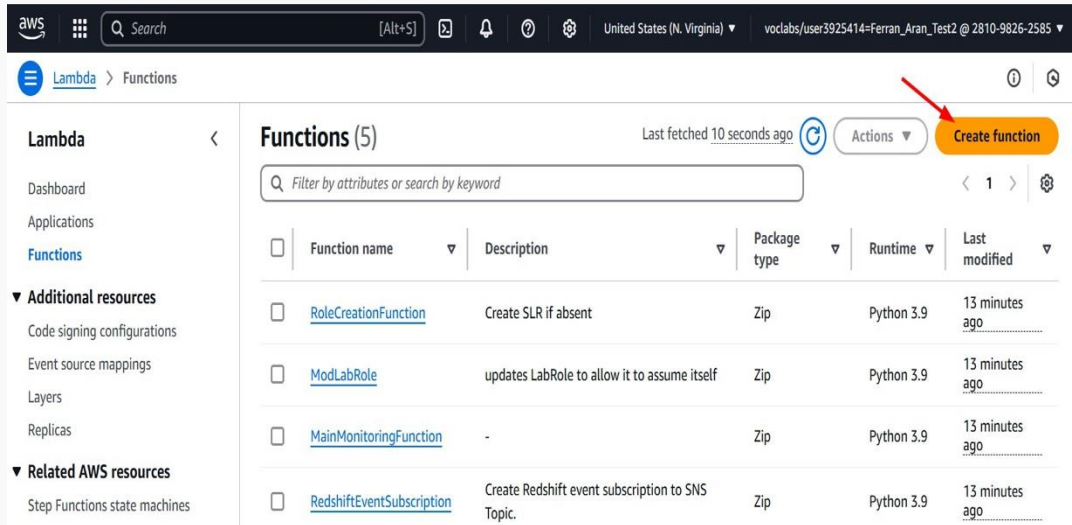


Figure 6: Lambda search

Step 2: Create the Lambda function

Now click on create function.



The screenshot shows the AWS Lambda console interface. At the top, there's a navigation bar with the AWS logo, a search bar, and various icons. Below the navigation bar, the left sidebar shows the 'Lambda' menu with options like 'Dashboard', 'Applications', and 'Functions'. The main content area is titled 'Functions (5)' and includes a search bar and a 'Create function' button highlighted with a red arrow. Below this, there's a table listing existing functions.

☐	Function name	Description	Package type	Runtime	Last modified
☐	RoleCreationFunction	Create SLR if absent	Zip	Python 3.9	13 minutes ago
☐	ModLabRole	updates LabRole to allow it to assume itself	Zip	Python 3.9	13 minutes ago
☐	MainMonitoringFunction	-	Zip	Python 3.9	13 minutes ago
☐	RedshiftEventSubscription	Create Redshift event subscription to SNS Topic.	Zip	Python 3.9	13 minutes ago

Figure 7: Create function

Step 2: Create the Lambda function

And fill the form like shown below (the function name doesn't matter but I suggest you use `count-cells`):

The screenshot shows the AWS Lambda 'Create function' page. At the top, the AWS navigation bar includes the logo, a search bar, and account information (United States (N. Virginia), Account ID: 7977-5604-8210, voclabs/user5018903=). The breadcrumb trail is 'Lambda > Functions > Create function'.

Create function [Info](#)

Choose one of the following options to create your function.

- ☒ **Author from scratch**
Start with a simple Hello World example.
- ☐ **Use a blueprint**
Build a Lambda application from sample code and configuration presets for common use cases.
- ☐ **Container image**
Select a container image to deploy for your function.

Basic information

Function name
Enter a name that describes the purpose of your function.

Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).

Runtime [Info](#)
Choose the language to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.

Permissions
By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. To use a different role, choose **Custom execution role** under **More settings**.

Custom settings [Info](#)

- Durable execution** - [new](#)
Build resilient, stateful applications with automatic failure recovery. ☐
- EC2 capacity provider** - [new](#)
Run Lambda on your preferred EC2 instance types instead of Lambda's serverless compute (default). ☐

Step 2: Create the Lambda function

And click on **Additional settings** and select **LabRole** and finally **Create function**:

General [Info](#)

ARM64 architecture ☐
Use ARM64 instead of x86_64 (default).

Custom execution role [Edit](#) ☒
Use custom execution role instead of new role with basic Lambda permissions (default).
IAM permissions
[LabRole](#)

Networking [Info](#)

Function URL ☐
Assign HTTP(S) endpoints.

Tenant isolation mode - new ☐
Guarantee separate execution environments for each tenant invoking the function.

VPC ☐
Connect to a VPC to securely access private resources.

Security & governance [Info](#)

Code signing ☐
Ensure only trusted code is deployed.

Encryption with AWS KMS ☐
Use your own KMS key to encrypt deployment package.

Tags ☐
Organize your function using labels.

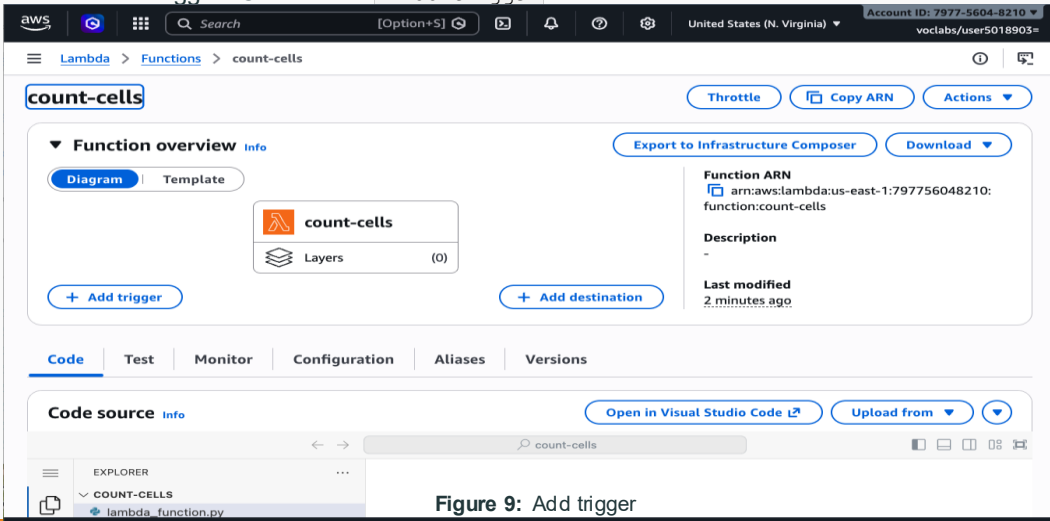
☐ Cannot add or remove after creation

[Cancel](#) [Create function](#)

Figure 8: Create function

Step 3: Add a trigger to the Lambda function

If we want our Lambda function to be executed when a new image is uploaded to the input bucket, we need to add a trigger. Click on the `+ Add trigger` button.



The screenshot displays the AWS Lambda console interface for a function named 'count-cells'. The top navigation bar includes the AWS logo, account ID (7977-5604-8210), and region (United States (N. Virginia)). The breadcrumb trail shows 'Lambda > Functions > count-cells'. The function name 'count-cells' is highlighted in a blue box. On the right, there are buttons for 'Throttle', 'Copy ARN', and 'Actions'. Below the function name, the 'Function overview' section is active, showing a 'Diagram' tab and a 'Template' tab. A diagram of the function is displayed, with a '+ Add trigger' button below it. To the right of the diagram, the 'Function ARN' is shown as 'arn:aws:lambda:us-east-1:797756048210:function:count-cells', and the 'Description' is empty. The 'Last modified' time is '2 minutes ago'. Below the overview section, there are tabs for 'Code', 'Test', 'Monitor', 'Configuration', 'Aliases', and 'Versions'. The 'Code source' section is active, showing a search bar and a '+ Add trigger' button. The bottom of the screenshot shows a file explorer with the file 'lambda_function.py' selected.

count-cells

Throttle Copy ARN Actions

Function overview Info

Diagram Template

count-cells

Layers (0)

+ Add trigger

+ Add destination

Export to Infrastructure Composer Download

Function ARN
arn:aws:lambda:us-east-1:797756048210:
function:count-cells

Description
-

Last modified
2 minutes ago

Code Test Monitor Configuration Aliases Versions

Code source Info

Open in Visual Studio Code Upload from

count-cells

EXPLORER

COUNT-CELLS

lambda_function.py

Figure 9: Add trigger

Step 3: Add a trigger to the Lambda function

Start by searching for **S3** in the trigger configuration and then fill in the form like shown below:

The screenshot shows the 'Add trigger' configuration page in the AWS Lambda console. The page is titled 'Add trigger' and has a sub-header 'Trigger configuration'. The configuration is for an S3 trigger. The bucket is 's3/medical-images-raw-ferran-aran'. The event types are 'All object create events'. The prefix is 'e.g. images/' and the suffix is 'e.g. .jpg'. The recursive invocation checkbox is checked. The page has a 'Cancel' button and an 'Add' button.

Trigger configuration [info](#)

S3
aws asynchronous storage

Bucket
Choose or enter the ARN of an S3 bucket that serves as the event source. The bucket must be in the same region as the function.
s3/medical-images-raw-ferran-aran
Bucket region: us-east-1

Event types
Select the events that you want to have trigger the Lambda function. You can optionally set up a prefix or suffix for an event. However, for each bucket, individual events cannot have multiple configurations with overlapping prefixes or suffixes that could match the same object key.
All object create events

Prefix - optional
Enter a single optional prefix to limit the notifications to objects with keys that start with matching characters. Any special characters must be URL encoded.
e.g. images/

Suffix - optional
Enter a single optional suffix to limit the notifications to objects with keys that end with matching characters. Any special characters must be URL encoded.
e.g. .jpg

Recursive invocation
If your function writes objects to an S3 bucket, ensure that you are using different S3 buckets for input and output. Writing to the same bucket increases the risk of creating a recursive invocation, which can result in increased Lambda usage and increased costs. [Learn more](#)
☒ I acknowledge that using the same S3 bucket for both input and output is not recommended and that this configuration can cause recursive invocations, increased Lambda usage, and increased costs.

Lambda will add the necessary permissions for AWS S3 to invoke your Lambda function from this trigger. [Learn more](#) about the Lambda permissions model.

[Cancel](#) [Add](#)

Figure 10: Add trigger

Step 3: Add a trigger to the lambda function

Okay so we've now configured our lambda to be triggered when a new object is created in the input bucket. But how do we access the image in the bucket from the lambda function?

Since we have configured the trigger to be an S3 event, AWS is going to send a **JSON object** to the lambda function with the information about the event.

Go back to the code tab as shown below.

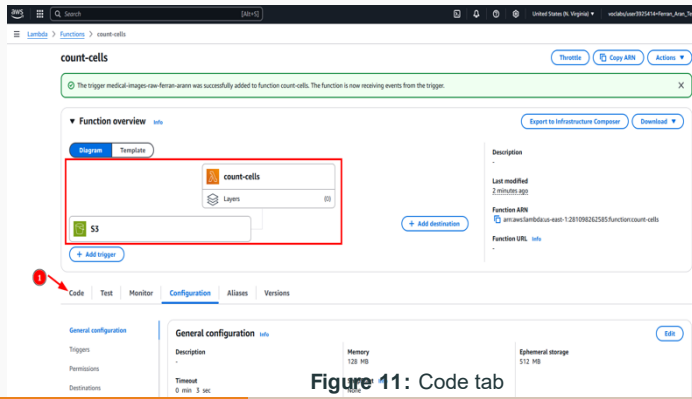


Figure 11: Code tab

Step 3: Add a trigger to the lambda function

Take a look at the default code that came with our lambda function:

```
import json

def lambda_handler(event, context):
    # TODO implement
    return {
        'statusCode': 200,
        'body': json.dumps('Hello from Lambda!')
    }
```

This is the basic structure of a lambda function. The `lambda_handler` function is the entry point of the lambda and it receives two arguments: `event` and `context`. The `event` argument is the JSON object we were talking about that contains the information about the event that triggered the lambda. So **anything we want to do with our lambda function has to be done inside this function.**

Step 3: Add a trigger to the lambda function

Let's see how the event looks like by printing it:

```
import json

def lambda_handler(event, context):
    print(json.dumps(event, indent=2))
    return {
        'statusCode': 200,
        'body': json.dumps('Hello from Lambda!')
    }
```

Step 3: Add a trigger to the lambda function

Once we are happy with the code, we need to “save” the changes by clicking on the `Deploy` button as shown below.

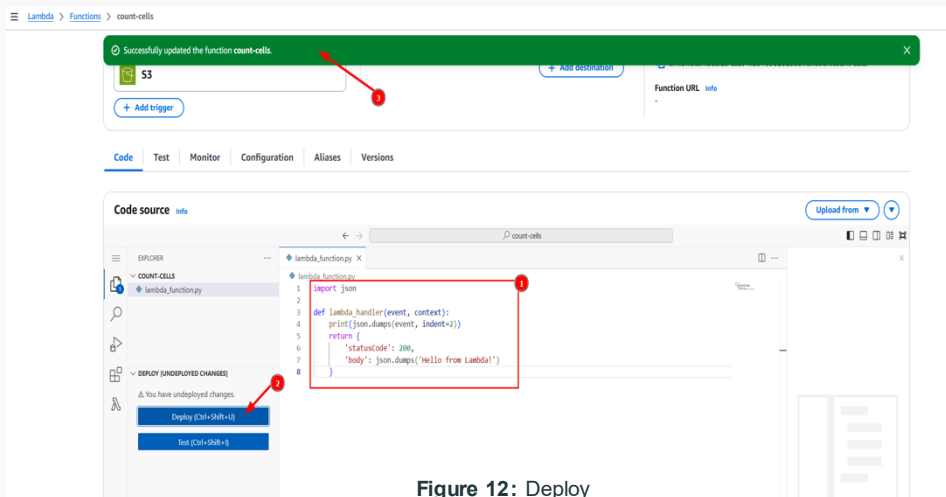


Figure 12: Deploy

Step 3: Add a trigger to the lambda function

Now we have to trigger the lambda function by uploading an image to the input bucket. You can do this by visiting the S3 service in the AWS Console and clicking on the input bucket `medical-images-raw-[YOUR-NAME]` we created earlier. Then click on `Upload` and select an image to upload.

We can now go back our lambda, click on `Monitor` and then on `View CloudWatch logs` to see the logs of the lambda function.

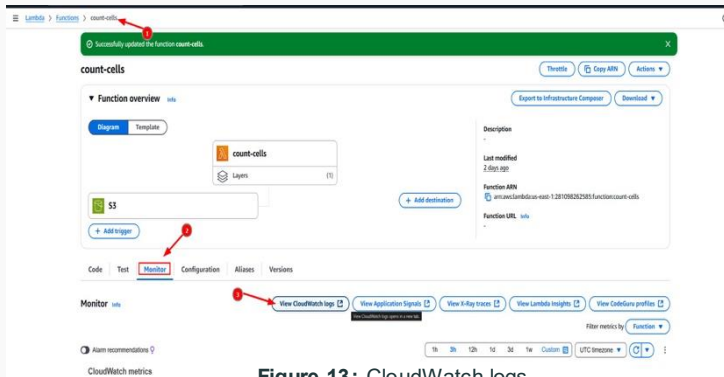


Figure 13: CloudWatch logs

Step 3: Add a trigger to the lambda function

Click on the latest log stream to see the logs of the lambda function.

The screenshot shows the AWS CloudWatch console interface. The breadcrumb navigation at the top indicates the path: CloudWatch > Log groups > /aws/lambda/count-cells. The left-hand navigation pane shows various CloudWatch features, with 'Logs' and 'Log groups' highlighted. The main content area is titled '/aws/lambda/count-cells' and contains a 'Log group details' section. This section provides information about the log group, including its class (Standard), ARN, creation time (2 days ago), retention (Never expire), and stored bytes (4.31 KB). To the right of these details are sections for 'Metric filters', 'Subscription filters', 'Contributor Insights rules', 'KMS key ID', 'Anomaly detection', 'Data protection', 'Sensitive data count', 'Field indexes', and 'Transformer'. Below the details section is a horizontal tab bar with 'Log streams' selected. The 'Log streams (1)' section displays a single log stream with the name '2025/03/18/[\$LATEST]4b2d42ce5b5c41c09d0a55f0312c1e61'. A red arrow points to this log stream name. The 'Last event time' for this stream is '2025-03-18 08:11:18 (UTC)'. At the top right of the console, there are buttons for 'Actions', 'View in Logs Insights', 'Start tailing', and 'Search log group'.

CloudWatch > Log groups > /aws/lambda/count-cells

/aws/lambda/count-cells

Log group details

- Log class** [Info](#)
Standard
- ARN**
[arn:aws:logs:us-east-1:281098262585:log-group:/aws/lambda/count-cells:](#)
- Creation time**
2 days ago
- Retention**
Never expire
- Stored bytes**
4.31 KB
- Metric filters**
0
- Subscription filters**
0
- Contributor Insights rules**
-
- KMS key ID**
-
- Anomaly detection**
[Configure](#)
- Data protection**
-
- Sensitive data count**
-
- Field indexes**
[Configure](#)
- Transformer**
[Configure](#)

Log streams (1)

☐ Exact match ☐ Show expired [Info](#) < 1 > [Info](#)

☐	Log stream	Last event time
☐	<u>2025/03/18/[\$LATEST]4b2d42ce5b5c41c09d0a55f0312c1e61</u>	2025-03-18 08:11:18 (UTC)

Figure 14: CloudWatch logs

Step 3: Add a trigger to the lambda function

If everything went well you should see the event printed in the logs in the form of a JSON. There is lots of information but we are just interested in a couple of fields; the S3 bucket name and the object key.

2025-03-18T08:11:18.800Z	},
2025-03-18T08:11:18.800Z	"s3": {
2025-03-18T08:11:18.800Z	"s3SchemaVersion": "1.0",
2025-03-18T08:11:18.800Z	"configurationId": "f22958d6-15e6-4a93-b535-c2cd60c26f04",
2025-03-18T08:11:18.800Z	"bucket": {
2025-03-18T08:11:18.800Z	"name": "medical-images-raw-ferran-arann",
2025-03-18T08:11:18.800Z	"ownerIdentity": {
2025-03-18T08:11:18.800Z	"principalId": "AIE480VMJFCTZ"
2025-03-18T08:11:18.800Z	},
2025-03-18T08:11:18.800Z	"arn": "arn:aws:s3:::medical-images-raw-ferran-arann"
2025-03-18T08:11:18.800Z	},
2025-03-18T08:11:18.800Z	"object": {
2025-03-18T08:11:18.800Z	"key": "image-1.png",
2025-03-18T08:11:18.800Z	"size": 133675,
2025-03-18T08:11:18.800Z	"eTag": "e326aa977c8ba59ab4f3c943d0b1cb03",
2025-03-18T08:11:18.800Z	"sequencer": "0067D92AA399156004"
2025-03-18T08:11:18.800Z	}

Figure 15: CloudWatch logs

Step 3: Add a trigger to the lambda function

Okay now that we know we have the information we need to access the image in the S3 bucket, we can write some template code that accesses the image on the bucket that triggered the lambda and saves it to the `processed` bucket.

We'll be using the `boto3` library to interact with S3 as we did in the previous session. Go back to your lambda function on the "Code" tab and paste the following code. **Remember to change the bucket name** (note that the code takes 2 slides to fit):

```
import boto3
import json
import os
import urllib.parse

s3 = boto3.client('s3')
```

Step 3: Add a trigger to the lambda function

```
def lambda_handler(event, context):  
    # Extract bucket and image info from the S3 event  
    bucket = event['Records'][0]['s3']['bucket']['name']  
    key = urllib.parse.unquote_plus(event['Records'][0]['s3']['object']['key'])  
  
    # Download the image from raw S3  
    download_path = f'/tmp/{os.path.basename(key)}'  
    s3.download_file(bucket, key, download_path)  
  
    # Upload the image to processed S3 bucket  
    result_bucket = 'medical-images-processed-[YOUR-NAME]' # Replace with your bucket name  
    s3.upload_file(download_path, result_bucket, key + "-processed.png")  
  
    return {  
        'statusCode': 200,  
        'body': json.dumps(f"Processed {key}, found {cell_count} cells.")  
    }
```

Step 3: Add a trigger to the lambda function

Once again click on `Deploy` to save the changes, then go to the S3 bucket `medical-images-raw-[YOUR-NAME]` and upload an image to trigger the lambda function.

If everything went well you should see the same image uploaded to the `processed` bucket with the suffix `-processed.png` as shown below:

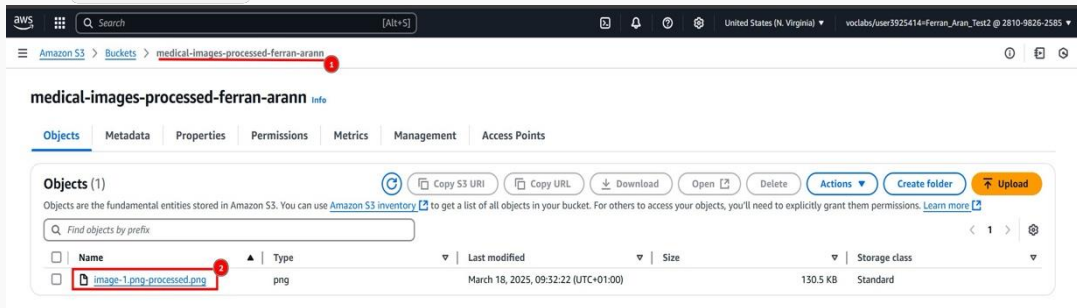


Figure 16: Processed image

Step 3: Add a trigger to the lambda function

Great, so we now have a lambda function that:

1. Is triggered when an image is uploaded to the input bucket.
2. Downloads the image from the input bucket.
3. Uploads the image to the output bucket.

We are now going to design the code that processes the image to count the cells, and once we're happy with it we'll add it to the lambda function code.

Step 4: Write the Lambda function code

Lets open up the remote jupyter notebook on our EC2 instance that we have been using and create a new notebook. As a reminder, you can access the EC2 instance, activate the python environment (in this case we are using the sample `project2` environment we created in Session 3) and start the jupyter notebook server with the following commands:

```
$ ssh -i .ssh/aws-keypair ec2-user@<your-ec2-public-ip>
[ec2]$ cd project2
[ec2]$ source .project2-venv/bin/activate
[ec2]$ jupyter notebook --ip 0.0.0.0 --port 8888
```

Remember you can visit the [second guide](#) on the subject's website to see how to access the notebook server.

Step 4: Write the Lambda function code

With the cell images downloaded and extracted, we can now upload one of them to the notebook server from the browser to start working on the code for the Lambda function.

To upload an image to the notebook server, just drag and drop it to the browser window as shown below:

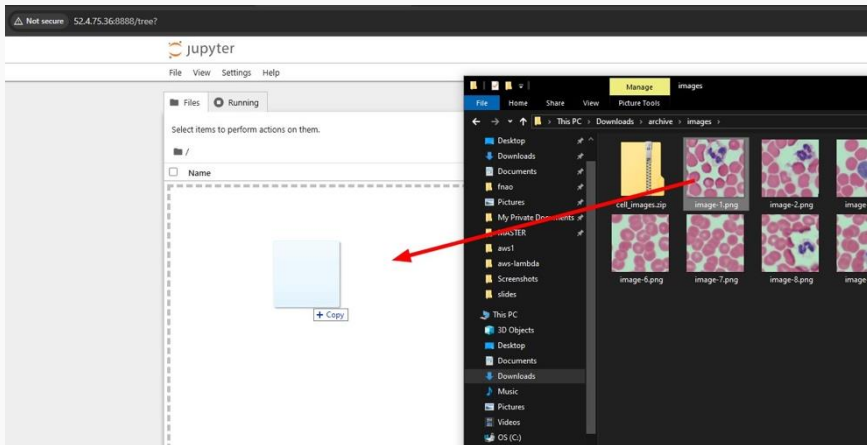


Figure 17: Upload image

Step 4: Write the Lambda function code

Next create a new notebook and paste the following code to install the dependencies.

```
!pip install matplotlib opencv-python  
!sudo dnf install mesa-libGL -y
```

And paste the following in another notebook to load the image and display it.

```
import cv2  
import matplotlib.pyplot as plt  
  
# Load the sample image  
image_path = 'image-1.png' # Replace with your image path  
image = cv2.imread(image_path)  
  
# Display the image  
plt.imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))  
plt.show()
```

Step 4: Write the Lambda function code

We are now free to work on whichever code we want to process the images. By using the Jupyter notebook we can test the code and see the results before deploying it to the Lambda function. For now, trust me and add the following code to the same notebook:

```
# Count cells by drawing contours around them
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
adaptive_thresh = cv2.adaptiveThreshold(
    gray_image, 255, cv2.ADAPTIVE_THRESH_MEAN_C,
    cv2.THRESH_BINARY_INV, 65, 5
)
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5,5))
morph_image = cv2.morphologyEx(adaptive_thresh, cv2.MORPH_OPEN, kernel, iterations=1)
contours, _ = cv2.findContours(
    morph_image, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
)

# Print the result
cell_count = len(contours)
print(f'Cell count: {cell_count}')
```

Step 4: Write the Lambda function code

Printing the result is fine but it would be even better if we could visualize the contours drawn around the cells. To do so, we can use the following code:

```
output_image = image.copy()
cv2.drawContours(output_image, contours, -1, (0, 255, 0), 2)

# Generate the images
fig, axes = plt.subplots(1, 2, figsize=(12, 6))

axes[0].imshow(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
axes[0].set_title('Original Image')
axes[0].axis('off')

axes[1].imshow(cv2.cvtColor(output_image, cv2.COLOR_BGR2RGB))
axes[1].set_title(f'Contours (Cells: {cell_count})')
axes[1].axis('off')

plt.show()
```

Step 4: Write the Lambda function code

The visualization should look like this:



Figure 18: Processed image

Step 4: Write the Lambda function code

By now our code does the following:

1. Load an image.
2. Process the image to count the cells.
3. Generate an image with the results.

We are now going to need to adapt this code to work in the Lambda function where it will have to read the image from the S3 bucket given its path and write the results back to another S3 bucket.

Step 4: Write the Lambda function code

In the AWS Console go to the Lambda service and click on the lambda function we created earlier, then scroll down to the code editor and paste the following code (note that the code takes 3 slides to fit):

```
import boto3
import cv2
import numpy as np
import json
import os
import urllib.parse

s3 = boto3.client('s3')

def lambda_handler(event, context):
    # Extract bucket and image info from the S3 event
    bucket = event['Records'][0]['s3']['bucket']['name']
    key = urllib.parse.unquote_plus(event['Records'][0]['s3']['object']['key'])
    original_name = os.path.splitext(os.path.basename(key))[0]

    download_path = f'/tmp/{os.path.basename(key)}'
```

Step 4: Write the Lambda function code

```
# Download and load the image from S3
s3.download_file(bucket, key, download_path)
image = cv2.imread(download_path)

# Count the cells using contours
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
adaptive_thresh = cv2.adaptiveThreshold(
    gray_image, 255, cv2.ADAPTIVE_THRESH_MEAN_C,
    cv2.THRESH_BINARY_INV, 65, 5
)
kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5,5))
morph_image = cv2.morphologyEx(adaptive_thresh, cv2.MORPH_OPEN, kernel, iterations=1)
contours, _ = cv2.findContours(
    morph_image, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
)
cell_count = len(contours)
```

Step 4: Write the Lambda function code

```
# Draw contours on a copy of the image
output_image = image.copy()
cv2.drawContours(output_image, contours, -1, (0, 255, 0), 2)

# Save the processed image and upload it to the "processed" bucket
result_image_name = f"{original_name}-processed-{cell_count}-cells.png"
result_image_path = f'/tmp/{result_image_name}'
cv2.imwrite(result_image_path, output_image)
result_bucket = 'medical-images-processed-[YOUR-NAME]' # Replace with your bucket name
s3.upload_file(result_image_path, result_bucket, result_image_name)

return {
    'statusCode': 200,
    'body': json.dumps(f"Processed {key}, found {cell_count} cells.")
}
```

Step 4: Write the Lambda function code

Once the code is copied click on Deploy to save the changes.

The screenshot displays the AWS Lambda console interface for a function named "count-cells". The left-hand sidebar contains the "EXPLORER" view, which shows the project structure with "COUNT-CELLS" expanded, containing the file "lambda_function.py". Below the explorer is the "DEPLOY" section, featuring two buttons: "Deploy (Ctrl+Shift+U)" and "Test (Ctrl+Shift+I)". A red arrow points to the "Deploy" button. The main area of the console shows the code editor for "lambda_function.py". The code defines a "lambda_handler" function that takes an event and context as input. It performs the following steps: 1. Defines a kernel for morphological operations. 2. Applies an adaptive threshold and morphological opening to the input image. 3. Finds contours in the processed image. 4. Counts the number of contours. 5. Draws the contours on a copy of the original image. 6. Saves the processed image to a local path. 7. Uploads the processed image to an S3 bucket named "medical-images-processed-ferran-arann".

```
10 def lambda_handler(event, context):
27     # Define a kernel for morphological operations
28     kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (5,5))
29     morph_image = cv2.morphologyEx(adaptive_thresh, cv2.MORPH_OPEN, kernel, iterations=1)
30     contours, _ = cv2.findContours(
31         morph_image, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
32     )
33     cell_count = len(contours)
34
35     # Draw contours on a copy of the image
36     output_image = image.copy()
37     cv2.drawContours(output_image, contours, -1, (0, 255, 0), 2)
38
39     # Save the processed image and upload it to the "processed" bucket
40     result_image_name = f'{original_name}-processed-{cell_count}-cells.png'
41     result_image_path = f'/tmp/{result_image_name}'
42     cv2.imwrite(result_image_path, output_image)
43     result_bucket = 'medical-images-processed-ferran-arann'
44     s3.upload_file(result_image_path, result_bucket, result_image_name)
45
```

Figure 19: Deploy

Step 5: Create and publish a Lambda layer with the dependencies

AWS Lambdas come with some python dependencies pre-installed such as `boto3` (which is the library we use to write and read from S3 buckets), but we are going to need to install `opencv-python` to process the images since it is not included by default.

To do so, we are going to create a Lambda layer with the dependencies and attach it to the Lambda function. Think of it as a way of packaging the needed dependencies so the lambda has them available when it runs.

We are going to need a machine with AWS CLI and its credentials configured as well as Python 3.13 and `zip` installed. I am going to use the EC2 instance we created in the previous unit together with `uv` for managing python versions. AWS CLI and `zip` are already installed in the instance.

Step 5: Create and publish a Lambda layer with the dependencies

Start by creating a folder which we'll use to build the layer and cd into it.

```
[ec2]$ mkdir -p cell-count-layer/python/lib/python3.14/site-packages/  
[ec2]$ cd cell-count-layer
```

Now create a virtual environment with `uv` and install the dependencies.

```
[ec2]$ uv venv --seed --python 3.13 .cell-count-venv source .cell-count-  
venv/bin/activate  
[ec2]$ pip install opencv-python-headless -t python/lib/python3.14/site-packages
```

Step 5: Create and publish a Lambda layer with the dependencies

Now we are going to zip the contents of the folder to create the layer.

```
[ec2]$ zip -r opencv.zip python
```

We'll need to create a bucket where we upload the layer so we can then import it to Lambda layers. You can use the AWS Console on your browser as we've done before or use the following command **where you have to replace [YOUR-NAME] with your name:**

```
[ec2]$ aws s3 mb s3://layers-bucket-[YOUR-NAME]
```

Step 5: Create and publish a Lambda layer with the dependencies

Now publish the layer to the bucket.

```
[ec2]$ aws s3 cp opencv.zip s3://layers-bucket-[YOUR-NAME] /
```

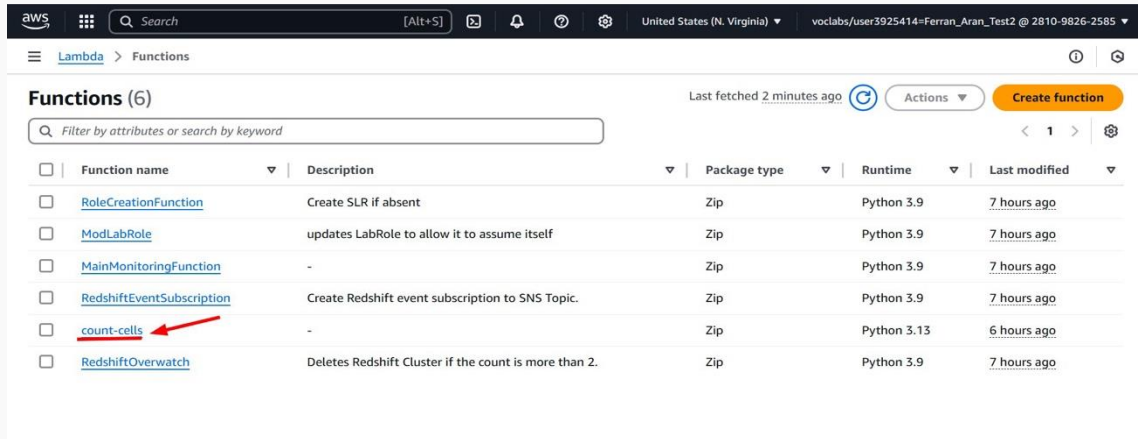
And finally, we are going to import the layer from the bucket to the Lambda layers.

```
[ec2]$ aws lambda publish-layer-version \  
  --layer-name opencv \  
  --content S3Bucket=layers-bucket-[YOUR-NAME],S3Key=opencv.zip \  
  --compatible-runtimes python3.14
```

Let's now see how to add this layer to our lambda.

Step 5: Create and publish a Lambda layer with the dependencies

Visit the Lambda service on the AWS Console and look for the function we have been working on. Click on it.



The screenshot shows the AWS Lambda console interface. At the top, there's a navigation bar with the AWS logo, a search bar, and various utility icons. Below the navigation bar, the breadcrumb trail shows 'Lambda > Functions'. The main heading is 'Functions (6)', and it indicates 'Last fetched 2 minutes ago'. There's a search bar with the placeholder text 'Filter by attributes or search by keyword'. To the right of the search bar are 'Actions' and 'Create function' buttons. Below these elements is a table listing the functions. The table has columns for 'Function name', 'Description', 'Package type', 'Runtime', and 'Last modified'. The 'count-cells' function is highlighted with a red arrow.

☐	Function name	Description	Package type	Runtime	Last modified
☐	RoleCreationFunction	Create SLR if absent	Zip	Python 3.9	7 hours ago
☐	ModLabRole	updates LabRole to allow it to assume itself	Zip	Python 3.9	7 hours ago
☐	MainMonitoringFunction	-	Zip	Python 3.9	7 hours ago
☐	RedshiftEventSubscription	Create Redshift event subscription to SNS Topic.	Zip	Python 3.9	7 hours ago
☐	count-cells	-	Zip	Python 3.13	6 hours ago
☐	RedshiftOverwatch	Deletes Redshift Cluster if the count is more than 2.	Zip	Python 3.9	7 hours ago

Figure 20: Lambda layer

Step 5: Create and publish a Lambda layer with the dependencies

Click on the **Layers** section below the function's name.

The screenshot displays the AWS Lambda console for a function named 'count-cells'. The breadcrumb navigation shows 'Lambda > Functions > count-cells'. The function's name 'count-cells' is prominently displayed at the top left. Below it, the 'Layers' section is highlighted with a red arrow, indicating where to click to manage dependencies. The 'Layers' section currently shows '(0)' layers. To the left of the function name, there is a diagram showing an S3 trigger. The right sidebar contains metadata: 'Description' (empty), 'Last modified' (1 second ago), 'Function ARN' (arn:aws:lambda:us-east-1:123456789012:function:count-cells), and 'Function URL' (Info). The bottom navigation bar includes tabs for 'Code', 'Test', 'Monitor', 'Configuration', 'Aliases', and 'Versions'. The 'Code' tab is currently selected.

Step 5: Create and publish a Lambda layer with the dependencies

Click on Edit

The screenshot displays the AWS Lambda console interface for a function named 'count-cells'. The top navigation bar includes the AWS logo, a search bar, and account information (United States (N. Vir), Account ID: 7977-5604-8210, voclabs/user5018903=).

The main content area is divided into two tabs: 'Runtime settings' (selected) and 'Edit runtime management configuration'. The 'Runtime settings' tab shows the following configuration:

- Runtime:** Python 3.14
- Handler:** lambda_function.lambda_handler
- Architecture:** x86_64

Below these settings is the 'Runtime management configuration' section, which includes:

- Runtime version ARN:** arn:aws:lambda:us-east-1::runtime:06dae9ab05ccfc05b6ef62a6c3f8c3976f2962ec872732763a914c0041151809
- Update runtime version:** Auto

The 'Layers (0)' section is currently empty, showing a search bar with the placeholder text 'Search by attributes or search by keyword' and a table header with the following columns: Merge order, Name, Layer version, Compatible runtimes, Compatible architectures, and Version ARN. A message below the table states 'There is no data to display.'

The bottom of the console features a footer with links for 'CloudShell', 'Feedback', 'Privacy', 'Terms', and 'Cookie preferences', along with a copyright notice: '© 2026, Amazon Web Services, Inc. or its affiliates.'

Figure 22: Lambda layer

Step 5: Create and publish a Lambda layer with the dependencies

Click on Add a Layer

The screenshot shows the AWS Lambda console interface. At the top, the navigation bar includes the AWS logo, a search bar, and account information (Account ID: 7977-5604-8210, voclabs/user5018903=). The breadcrumb trail indicates the path: Lambda > Functions > count-cells > Edit layers. The main content area is divided into two sections. The first section, 'Function runtime settings', displays the 'Runtime' as 'Python 3.14' and the 'Architecture' as 'x86_64'. The second section, 'Layers', includes an 'Info' link and a message stating 'No layers are configured on the function.' Below this message is a blue button labeled 'Add a layer'. A light blue informational box at the bottom of the main content area contains a note about layer compatibility with Lambda Managed Instances. The footer of the console shows links for 'CloudShell', 'Feedback', 'Privacy', 'Terms', and 'Cookie preferences', along with the copyright notice '© 2026, Amazon Web Services, Inc. or its affiliates.'

aws [Option+S] United States (N. Vir) Account ID: 7977-5604-8210 voclabs/user5018903=

≡ [Lambda](#) > [Functions](#) > [count-cells](#) > Edit layers ⓘ ↻

Function runtime settings

Runtime Python 3.14	Architecture x86_64
-------------------------------	-------------------------------

Layers [Info](#)

No layers are configured on the function.

[Add a layer](#)

Info Some Lambda layers may not be compatible with functions running on Lambda Managed Instances. Verify layer compatibility and remove any incompatible layers from your function configuration. Layer compatibility depends on the runtime and architecture of your EC2 instance.

CloudShell Feedback Privacy Terms Cookie preferences

© 2026, Amazon Web Services, Inc. or its affiliates.

Step 5: Create and publish a Lambda layer with the dependencies

Click on **Custom layers** and select the layer we just created. Finally click on **Add**.

The screenshot shows the 'Add layer' page in the AWS Lambda console. At the top, the breadcrumb 'Lambda > Add layer' is visible. The 'Function runtime settings' section shows 'Runtime' as 'Python 3.13' and 'Architecture' as 'x86_64'. The 'Choose a layer' section has three options: 'AWS layers', 'Custom layers' (selected), and 'Specify an ARN'. Below 'Custom layers', there is a list of layers. The first layer, 'opencv', is selected and highlighted with a red box and a red circle with the number '3'. Below it, the 'Version' is set to '1', also highlighted with a red box and a red circle with the number '5'. A red arrow with the number '1' points to the 'Custom layers' radio button. Another red arrow with the number '2' points to the 'Add' button at the bottom right. A third red arrow with the number '4' points to the 'Cancel' button at the bottom right. The 'Add' button is orange and labeled 'Add', while the 'Cancel' button is blue and labeled 'Cancel'.

Add layer

Function runtime settings

Runtime
Python 3.13

Architecture
x86_64

Choose a layer

Layer source [Info](#)
Choose from layers with a compatible runtime and instruction set architecture or specify the Amazon Resource Name (ARN) of a layer version. You can also [create a new layer](#).

☐ AWS layers
Choose a layer from a list of layers provided by AWS.

☒ Custom layers
Choose a layer from a list of layers created by your AWS account.

☐ Specify an ARN
Specify a layer by providing the ARN.

Custom layers
Layers created by your AWS account that are compatible with your function's runtime.

opencv

Version
1

Cancel Add

Figure 23: Lambda layer

Step 5: Create and publish a Lambda layer with the dependencies

Click on **Save**

The screenshot shows the AWS Lambda console interface. The breadcrumb navigation at the top indicates the path: **Lambda** > **Functions** > **count-cells** > **Edit layers**. The main content area is titled **Layers** with an **Info** link. Below the title, there is a list of layers. One layer, **opencv**, is shown with its ARN: `arn:aws:lambda:us-east-1:797756048210:layer:opencv:1`. To the left of the layer name is a grid icon, and to the right is a trash icon. Below the list is a button labeled **Add a layer**. A warning message in a light blue box states: "Some Lambda layers may not be compatible with functions running on Lambda Managed Instances. Verify layer compatibility and remove any incompatible layers from your function configuration. Layer compatibility depends on the runtime and architecture of your EC2 instance." At the bottom right of the console, there are two buttons: **Cancel** and **Save**. The footer of the console includes the **CloudShell** icon, a **Feedback** link, and links for **Privacy**, **Terms**, and **Cookie preferences**. The copyright notice at the very bottom reads: "© 2026, Amazon Web Services, Inc. or its affiliates."

Step 6: Upload the images to the input bucket

Remember the cell images can be found [here](#).

To upload them to the S3 bucket, we could do so by using the AWS Console as we did before, but this time we are going to use the AWS CLI to do it.

```
$ cd DIRECTORY_WITH_cell_images  
$ aws s3 cp ./cell_images s3://medical-images-raw-[YOUR-NAME]/ --recursive
```

The next slide contains a screenshot of the general steps to download, extract and upload the images to the S3 bucket.

Step 6: Upload the images to the input bucket

The screenshot shows a course page for 'UNIVERSITAT ROVIRA I VIRGILI' with a sidebar of course materials. The main content area displays a file explorer window showing the 'Downloads' folder, which contains a subfolder named 'cell_images'. A terminal window is open, showing the command to list the contents of the 'cell_images' directory and the subsequent command to upload the images to an AWS S3 bucket named 'medical-images-raw-ferran-arann'.

Course Materials:

- Linux Parallelism 1.4 MB Document PDF
- SLURM - Part I 3.9 MB Document PDF
- SLURM Part I video 200.3 MB Fitxer de vídeo (MP4)
- SLURM Part II 1.6 MB Document PDF
- Tema 3**
- Session 1 & 2 - Cloud theory 1.8 MB Document PDF
- Subject website with additional resources
- Lab 01: Deploying your personal website
- Session 3 - AWS EC2 (v2) 3.0 MB Document PDF
- Session 4 - AWS S3 7.4 MB Document PDF
- Cell images dataset (for Session 5 - AWS Lambda)** 1.2 MB Fitxer (ZIP)

File Explorer (Downloads):

- cell_images.zip
- cell_images

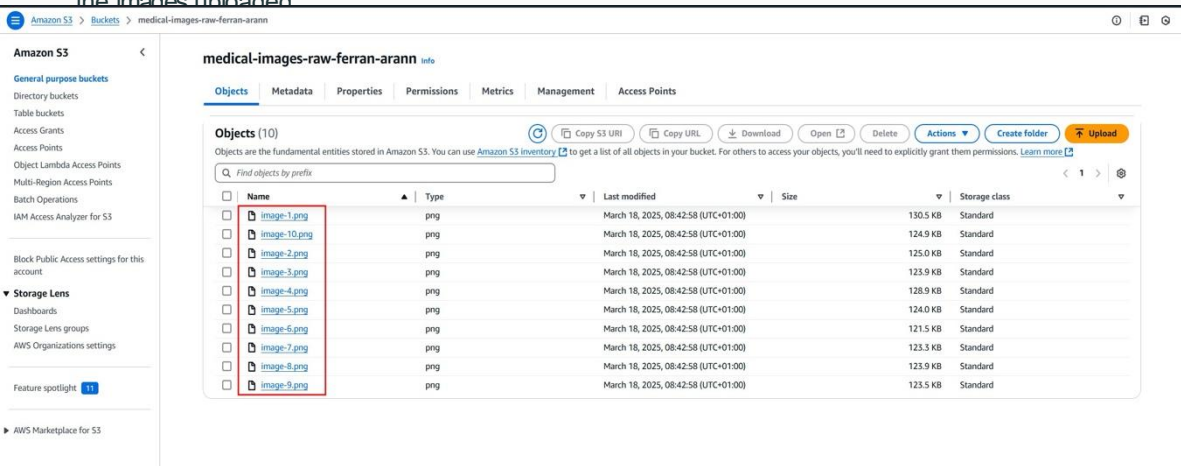
Terminal Output:

```
PS C:\Users\fnao\Downloads> ls .\cell_images\  
Directory: C:\Users\fnao\Downloads\cell_images  
  
Mode                LastWriteTime         Length Name  
----                -  
-a- 4/28/2020 2:20 AM      133675 image-1.png  
-a- 4/28/2020 2:20 AM      127869 image-10.png  
-a- 4/28/2020 2:20 AM      127968 image-2.png  
-a- 4/28/2020 2:20 AM      126862 image-3.png  
-a- 4/28/2020 2:20 AM      132020 image-4.png  
-a- 4/28/2020 2:20 AM      127001 image-5.png  
-a- 4/28/2020 2:20 AM      124000 image-6.png  
-a- 4/28/2020 2:20 AM      126204 image-7.png  
-a- 4/28/2020 2:20 AM      126892 image-8.png  
-a- 4/28/2020 2:20 AM      126418 image-9.png  
  
PS C:\Users\fnao\Downloads> aws s3 cp ./cell_images s3://medical-images-raw-ferran-arann/ --recursive  
upload: cell_images/image-1.png to s3://medical-images-raw-ferran-arann/image-1.png  
upload: cell_images/image-10.png to s3://medical-images-raw-ferran-arann/image-10.png  
upload: cell_images/image-2.png to s3://medical-images-raw-ferran-arann/image-2.png  
upload: cell_images/image-3.png to s3://medical-images-raw-ferran-arann/image-3.png  
upload: cell_images/image-4.png to s3://medical-images-raw-ferran-arann/image-4.png  
upload: cell_images/image-5.png to s3://medical-images-raw-ferran-arann/image-5.png  
upload: cell_images/image-6.png to s3://medical-images-raw-ferran-arann/image-6.png  
upload: cell_images/image-7.png to s3://medical-images-raw-ferran-arann/image-7.png  
upload: cell_images/image-8.png to s3://medical-images-raw-ferran-arann/image-8.png  
upload: cell_images/image-9.png to s3://medical-images-raw-ferran-arann/image-9.png  
upload: cell_images/image-10.png to s3://medical-images-raw-ferran-arann/image-10.png  
PS C:\Users\fnao\Downloads>
```

Figure 24: Upload images

Step 6: Upload the images to the input bucket

If we check on the AWS Console the input bucket `medical-images-raw-[YOUR-NAME]` we should see the images uploaded



The screenshot shows the AWS S3 console interface. The left sidebar contains navigation links for Amazon S3, including General purpose buckets, Directory buckets, Table buckets, Access Grants, Access Points, Object Lambda Access Points, Multi-Region Access Points, Batch Operations, IAM Access Analyzer for S3, Storage Lens, Dashboards, Storage Lens groups, AWS Organizations settings, Feature spotlight, and AWS Marketplace for S3. The main content area displays the bucket 'medical-images-raw-ferran-arann' with tabs for Objects, Metadata, Properties, Permissions, Metrics, Management, and Access Points. The 'Objects' tab is active, showing a list of 10 objects. A red box highlights the first five objects: image-1.png, image-10.png, image-2.png, image-3.png, and image-4.png. The table lists all 10 objects, each with a checkbox, name, type (png), last modified date (March 18, 2025, 08:42:58 (UTC+01:00)), size, and storage class (Standard).

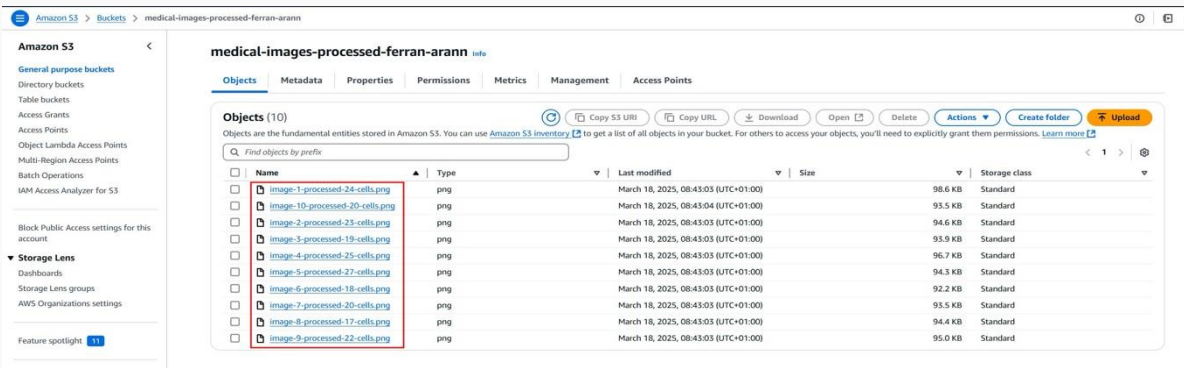
☐	Name	Type	Last modified	Size	Storage class
☐	image-1.png	png	March 18, 2025, 08:42:58 (UTC+01:00)	130.5 KB	Standard
☐	image-10.png	png	March 18, 2025, 08:42:58 (UTC+01:00)	124.9 KB	Standard
☐	image-2.png	png	March 18, 2025, 08:42:58 (UTC+01:00)	125.0 KB	Standard
☐	image-3.png	png	March 18, 2025, 08:42:58 (UTC+01:00)	123.9 KB	Standard
☐	image-4.png	png	March 18, 2025, 08:42:58 (UTC+01:00)	128.9 KB	Standard
☐	image-5.png	png	March 18, 2025, 08:42:58 (UTC+01:00)	124.0 KB	Standard
☐	image-6.png	png	March 18, 2025, 08:42:58 (UTC+01:00)	121.5 KB	Standard
☐	image-7.png	png	March 18, 2025, 08:42:58 (UTC+01:00)	123.3 KB	Standard
☐	image-8.png	png	March 18, 2025, 08:42:58 (UTC+01:00)	123.9 KB	Standard
☐	image-9.png	png	March 18, 2025, 08:42:58 (UTC+01:00)	123.5 KB	Standard

Figure 25: Uploaded images

Step 7: Check the results in the output bucket and verify the Lambda logs

If we've done everything correctly, a Lambda function should have been triggered for each image uploaded to the input bucket, and the processed images should be in the output bucket.

If we check S3 bucket `medical-images-processed-[YOUR-NAME]` we should see something like this:



The screenshot shows the Amazon S3 console interface. On the left is a navigation pane with 'Amazon S3' selected. The main area displays the bucket 'medical-images-processed-ferran-arann'. The 'Objects' tab is active, showing a list of 10 objects. A red rectangle highlights the first nine objects, which are all PNG files named 'image-1-processed-24-cells.png' through 'image-9-processed-22-cells.png'. The tenth object is 'image-10-processed-20-cells.png'. Each object entry includes a checkbox, a thumbnail icon, the name, type (png), last modified date (March 18, 2025, 08:43:03 (UTC+01:00)), size (ranging from 93.5 KB to 98.6 KB), and storage class (Standard).

☐	Name	Type	Last modified	Size	Storage class
☐	image-1-processed-24-cells.png	png	March 18, 2025, 08:43:03 (UTC+01:00)	98.6 KB	Standard
☐	image-10-processed-20-cells.png	png	March 18, 2025, 08:43:04 (UTC+01:00)	93.5 KB	Standard
☐	image-2-processed-23-cells.png	png	March 18, 2025, 08:43:03 (UTC+01:00)	94.6 KB	Standard
☐	image-3-processed-19-cells.png	png	March 18, 2025, 08:43:03 (UTC+01:00)	93.9 KB	Standard
☐	image-4-processed-25-cells.png	png	March 18, 2025, 08:43:03 (UTC+01:00)	96.7 KB	Standard
☐	image-5-processed-27-cells.png	png	March 18, 2025, 08:43:03 (UTC+01:00)	94.3 KB	Standard
☐	image-6-processed-18-cells.png	png	March 18, 2025, 08:43:03 (UTC+01:00)	92.2 KB	Standard
☐	image-7-processed-20-cells.png	png	March 18, 2025, 08:43:03 (UTC+01:00)	93.5 KB	Standard
☐	image-8-processed-17-cells.png	png	March 18, 2025, 08:43:03 (UTC+01:00)	94.4 KB	Standard
☐	image-9-processed-22-cells.png	png	March 18, 2025, 08:43:03 (UTC+01:00)	95.0 KB	Standard

Figure 26: Processed images

Step 7: Check the results in the output bucket and verify the Lambda logs

And if we go back to our lambda and click on **Monitoring**. We should see a plot named **Invocations** that shows the number of times the lambda has been triggered.

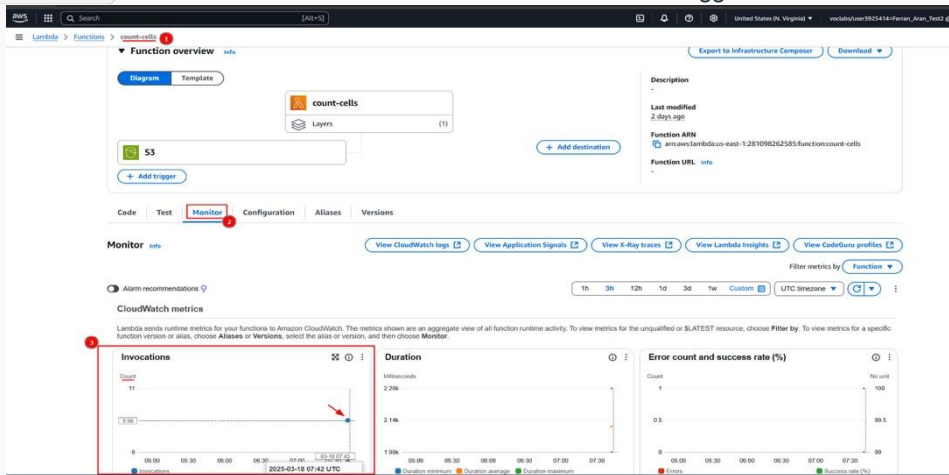


Figure 27: Lambda invocations

Step 7: Check the results in the output bucket and verify the Lambda logs

We could also click on [View logs in CloudWatch](#) to see the logs of the lambda function as we did earlier.

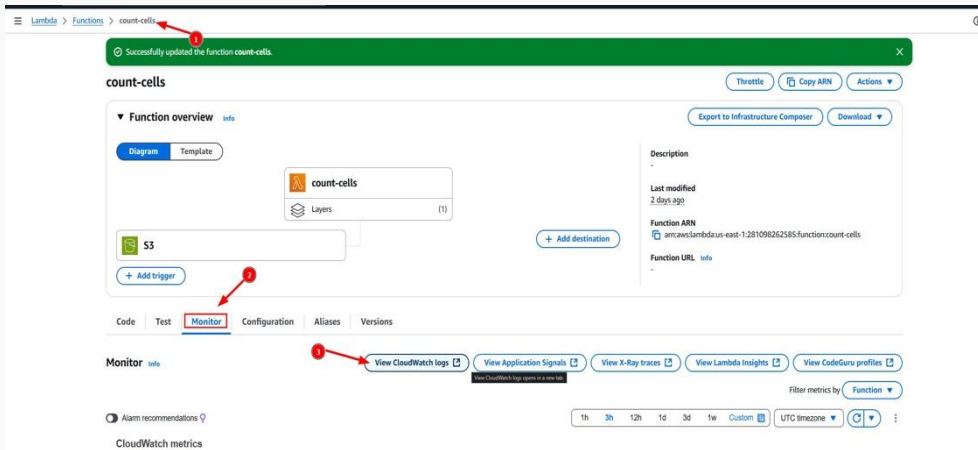


Figure 28: Lambda logs

Recap

- AWS Lambda is a serverless computing service that runs code in response to events.
- Lambda functions are triggered by events from various sources, such as S3, API Gateway, and CloudWatch.
- Lambda functions can be used for real-time data processing, data aggregation, data validation, and image processing in healthcare applications.